Laboratory – Logical Reasoning for Planning

Using an LLM to Construct and Test a Simple Planning Agent

Undergraduate Artificial Intelligence

Contents

1 Learning Objectives

By the end of this laboratory, you should be able to:

1. describe a simple planning problem using states, actions, preconditions, and effects;
2. determine whether an action is logically applicable in a given state;
3. explain how logical reasoning and search work together in planning;
4. use an LLM to generate a simple planning program;
5. test and validate LLM-generated code;
6. identify limitations of relying on an LLM to generate an intelligent agent.

The central idea of the laboratory is:

$$\boxed{\text{Logic} + \text{Search} = \text{Planning}}$$

2 Background

A planning problem asks an agent to find a sequence of actions that transforms an initial state into a desired goal state.

We can describe a planning problem using:

$$\boxed{(I, A, G)}$$

where

- I is the initial state;
- A is the set of available actions;
- G is the goal.

Each action has:

- **preconditions**: what must be true before the action can be performed;
- **effects**: what becomes true or false after the action is performed.

For example, consider the action

$$\text{Move}(A, B).$$

It might have the precondition

$$\text{At}(\mathit{Robot}, A)$$

and effects

$$\neg \mathrm{At}(Robot, A)$$

and

$$\mathrm{At}(Robot, B).$$

The action is therefore applicable only when the precondition is satisfied.

This gives an important connection with logical reasoning:

$$\boxed{S \models Preconditions(a)}$$

means that action a is applicable in state S.

3 Motivating Scenario

You are designing a small warehouse robot.

The robot must transport a package from one location to another.

The warehouse contains three locations:

$$A, \qquad B, \qquad C.$$

The robot starts at A, and the package is also at A.

The goal is to deliver the package to C.

Thus the initial state is:

$$I = \{\mathrm{At}(Robot, A), \mathrm{At}(Package, A)\}.$$

The goal is:

$$G = \{\mathrm{At}(Package, C)\}.$$

The robot has the following actions.

Move

The robot can move between connected locations.

For example:

$$\mathrm{Move}(A, B)$$

has precondition

$$\text{At}(Robot, A)$$

and effects

$$\neg\text{At}(Robot, A), \qquad \text{At}(Robot, B).$$

Similarly, the robot can move from B to A, B to C, and C to B.

Pick Up

The robot can pick up the package when both are at the same location.
For example:

$$\text{PickUp}(Package, A)$$

has preconditions

$$\text{At}(Robot, A)$$

and

$$\text{At}(Package, A).$$

Its effects are

$$\neg\text{At}(Package, A)$$

and

$$\text{Holding}(Package).$$

Drop

The robot can drop the package at its current location.
For example:

$$\text{Drop}(Package, C)$$

has preconditions

$$\text{At}(Robot, C)$$

and

$$\text{Holding}(Package).$$

Its effects are

$$\neg\text{Holding}(Package)$$

and

$$\text{At}(Package, C).$$

Task 0: Understand the Planning Problem

Before using an LLM, write down the planning problem.

(a) What is the initial state I?

(b) What is the goal G?

(c) List the actions available to the robot.

(d) For each action, identify its preconditions and effects.

Now determine which actions are initially applicable.

Question

Starting from

$$I = \{\text{At}(Robot, A), \text{At}(Package, A)\},$$

is the action

$$\text{PickUp}(Package, A)$$

applicable?
What about

$$\text{Drop}(Package, C)?$$

Explain your answers using the action preconditions.

Think About It

An action should not be considered applicable merely because it appears in the list of available actions.
Ask:

Are all of its preconditions satisfied in the current state?

This is where logical reasoning enters the planning process.

Task 1: Construct a Plan by Hand

Before writing any program, try to construct a valid plan manually.
A plan is a sequence of actions

$$a_1, a_2, \ldots, a_n$$

such that

$$I \xrightarrow{a_1} S_1 \xrightarrow{a_2} S_2 \to \cdots \to S_n$$

and

$$S_n \models G.$$

Find a sequence of actions that transports the package from A to C.
For example, you might need to reason about:

$$\text{Move}(A, B),$$

$$\text{PickUp}(Package, B),$$

$$\text{Move}(B, C),$$

and

$$\text{Drop}(Package, C).$$

Write down the state after every action.

Example format

State	Facts
S_0	$\text{At}(Robot, A)$, $\text{At}(Package, A)$
S_1	$\cdots$
S_2	$\cdots$
S_3	$\cdots$

Task 2: Ask an LLM to Implement the Planner

Now use a freely available LLM such as ChatGPT, Gemini, Claude, or Microsoft Copilot.
Do *not* immediately ask the LLM to "solve the problem."
Instead, give it a precise specification.
A suitable prompt is:

I want to implement a simple planning agent in Python.
Represent a state as a set of logical propositions.
Each action should contain:

- a name;
- positive preconditions;
- negative preconditions;
- positive effects;
- negative effects.

An action is applicable if all of its preconditions are satisfied by the current state.
When an action is applied:

1. remove its negative effects from the state;
2. add its positive effects to the state.

Use breadth-first search to find a sequence of actions that achieves a specified goal.
The program should also:

- detect when no plan exists;
- print the resulting sequence of actions;
- print the states reached after each action.

Explain the implementation and identify any assumptions you make.

Run the generated program on the warehouse problem.

Think About It

Before accepting the generated code, identify where the following ideas from the specification appear in the program:

Preconditions	$\longrightarrow$ When is an action applicable?
Effects	$\longrightarrow$ How does the state change?
Goal	$\longrightarrow$ When does planning terminate?
BFS	$\longrightarrow$ How are alternative plans explored?

Task 3: Test the Generated Planner

Do not assume that the LLM-generated program is correct.
Construct and run at least three tests.

Test A: Solvable Problem

Use the original warehouse problem.
The program should find a plan that achieves

$$\text{At}(Package, C).$$

Verify every action in the returned plan.

Test B: Impossible Problem

Modify the problem so that the robot cannot pick up the package.
For example, remove the `PickUp` action.
The planner should report:

$$\boxed{\text{No plan found}}$$

rather than inventing an action.

Test C: Irrelevant Actions

Add an action that moves the robot but does not move the package.
For example:

$$\text{Move}(A, B)$$

may be available even when the package remains at A.
Check that the planner does not incorrectly treat the robot reaching C as equivalent to the package reaching C.

Record Your Results

For each test, record:

- the initial state;
- the goal;
- whether a plan was found;
- the resulting plan;
- whether the plan is actually valid.

Task 4: Logic and Search

The planner uses two different ideas:

$$\boxed{\text{Logical reasoning}}$$

and

$$\boxed{\text{Search}}.$$

Explain where each is used.
Consider a state S and an action a.
The logical component determines whether

$$S \models Preconditions(a).$$

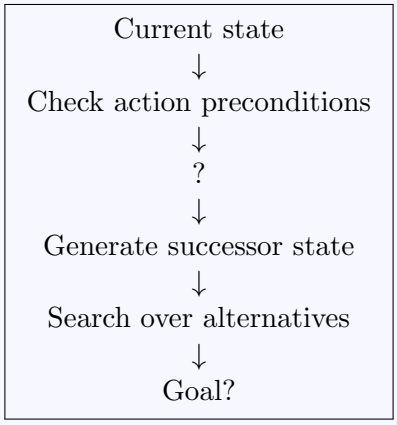
If this is true, the action can be applied to obtain a new state:

$$S' = Apply(S, a).$$

The search component then determines which sequence of applicable actions should be explored.

Question

Complete the following description:



Explain in your own words how logical reasoning and search work together to produce a plan.

Think About It

A useful way to think about the planner is:

Logic determines what is possible; search determines what to try.

This is closely related to the search methods studied in the previous module.

Task 5: Optional – Can the LLM Verify Its Own Plan?

Ask the LLM to explain why the plan it generated is valid.
For example, ask:

> For every action in the plan, identify its preconditions and show that those preconditions are satisfied in the state in which the action is executed.

Now compare the explanation with the actual state transitions computed by your Python program.

Question

Which should you trust more:

(a) the LLM's explanation;

(b) the independently executed state transitions?

Explain why.

Important

The purpose of this exercise is not to demonstrate that an LLM is unreliable.
It is to illustrate a general engineering principle:

A generated explanation is not the same as an independent verification.

4 Second Planning Domain: A Locked-Door Navigation Problem

The warehouse example combines movement with transporting an object. In this second domain, the agent must obtain a resource before it can access a location. Use the *same* state-transition and BFS machinery; change only the problem description.

There are three rooms A, B, C. The passages $A \leftrightarrow B$ and $B \leftrightarrow C$ are the only passages. The passage between B and C is initially locked. The robot starts in A; a key is in A. The goal is to reach C. The robot may carry the key, and unlocking the door permanently opens it. There is no action to drop the key or relock the door.

Represent states using the propositions

$$\mathrm{At}(A),\ \mathrm{At}(B),\ \mathrm{At}(C),\ \mathrm{KeyAt}(A),\ \mathrm{HasKey},\ \mathrm{Unlocked}.$$

Exactly one $\mathrm{At}(\cdot)$ proposition is true in a reachable state. The initial state and goal are

$$I = \{\mathrm{At}(A), \mathrm{KeyAt}(A)\}, \qquad G = \{\mathrm{At}(C)\}.$$

Action	Preconditions	Effects
$\mathrm{Move}(A, B)$	$\mathrm{At}(A)$	delete $\mathrm{At}(A)$; add $\mathrm{At}(B)$
$\mathrm{Move}(B, A)$	$\mathrm{At}(B)$	delete $\mathrm{At}(B)$; add $\mathrm{At}(A)$
PickKey	$\mathrm{At}(A), \mathrm{KeyAt}(A)$	delete $\mathrm{KeyAt}(A)$; add HasKey
Unlock	$\mathrm{At}(B)$, HasKey	add Unlocked
$\mathrm{Move}(B, C)$	$\mathrm{At}(B)$, Unlocked	delete $\mathrm{At}(B)$; add $\mathrm{At}(C)$
$\mathrm{Move}(C, B)$	$\mathrm{At}(C)$, Unlocked	delete $\mathrm{At}(C)$; add $\mathrm{At}(B)$

All unlisted effects are absent. A proposition not in the state is false. Actions have unit cost.

Task 6: Plan with a Key and a Locked Door

(a) By hand, construct a shortest valid plan from I to G. Show the state after each action.

(b) Reuse your BFS planner from the warehouse task. Encode these six actions and print the resulting plan and state trace.

(c) Remove PickKey from the action set. Does any plan exist? Explain.

(d) Validate the proposed sequence

$$\mathrm{Move}(A, B),\ \mathrm{Unlock},\ \mathrm{Move}(B, C).$$

Report the first invalid action and its unsatisfied precondition(s).

Task 7: Compare LLM Plans Across Two Domains

Without showing your BFS solution, ask an LLM to propose a plan for the locked-door problem. Provide the exact initial state, goal, and action table. Request only a sequence of the six permitted actions.
Then use your Python validator to check the proposed plan. Record whether it is valid, reaches the goal, and has the same length as a shortest BFS plan. If the plan fails, give the LLM the validator's first error and ask it to revise its plan. Run the validator again.

Think About It

Does a plausible narrative guarantee that an action's preconditions are satisfied? What can an independent state-transition checker establish that an LLM explanation cannot?

5 Submission

Submit:

1. your specification of the planning problem;
2. your manually constructed plan;
3. the prompt used with the LLM;
4. the generated Python program;
5. the results of your tests;
6. your answers to the "Think About It" questions;
7. your reflection on the use of the LLM;
8. your locked-door plan, test results, and LLM comparison.

You should clearly identify which parts of the program were generated or modified with the assistance of an LLM.

6 Reflection Questions

Answer the following questions briefly.

1. Why is it useful to specify action preconditions and effects before asking an LLM to write the planner?
2. Give an example of an error that could occur if the planner failed to check an action's preconditions.
3. Why is a plan that "looks reasonable" not necessarily a valid plan?

4. What did the LLM contribute to the implementation?
5. What did you have to verify independently?
6. In this laboratory, where is logical reasoning being used?
7. How is planning related to the search algorithms studied in the previous module?

7 Takeaway

The laboratory illustrates a useful combination of ideas:

$$\boxed{\text{Logical reasoning} + \text{Search} \rightarrow \text{Planning}}$$

The logical component determines whether actions are applicable and how they change the world.

The search component explores possible sequences of actions.

An LLM can assist with the engineering implementation, but the resulting agent must still be tested and validated.

More generally:

$$\boxed{\text{Understand} \rightarrow \text{Specify} \rightarrow \text{Generate} \rightarrow \text{Execute} \rightarrow \text{Verify}}$$

This is the central lesson of using LLMs as engineering tools for intelligent systems.

8 Optional Extension: Prolog as a Logical Verifier

Important

This section is optional.
It is intended as a short introduction to Prolog as an executable rule-based reasoning system.
You do not need to learn Prolog in depth to complete the exercise.

8.1 Why Prolog?

So far, we have represented logical knowledge and used Python to implement a planning agent.

Prolog provides another way of working with logical knowledge.

A Prolog program consists primarily of:

- **facts**;
- **rules**;
- **queries**.

For example:

```
penguin(polly).

bird(X) :-
    penguin(X).

animal(X) :-
    bird(X).
```

The first line is a fact:

$$\text{Penguin}(Polly).$$

The second line represents the rule

$$\text{Penguin}(x) \rightarrow \text{Bird}(x).$$

The third represents

$$\text{Bird}(x) \rightarrow \text{Animal}(x).$$

We can then ask Prolog:

```
?- animal(polly).
```

and Prolog should respond:

```
true.
```

The reasoning is:

$$\text{Penguin}(Polly) \Rightarrow \text{Bird}(Polly) \Rightarrow \text{Animal}(Polly).$$

Thus Prolog provides a simple example of

$$\boxed{\text{Facts} + \text{Rules} + \text{Inference.}}$$

Task 8: Prolog as a Plan Verifier

Install or access a Prolog implementation such as SWI-Prolog.
You can use a locally installed version or an online environment if available.
Create a file called

`planner.pl`

and enter the following facts describing the warehouse:

```
connected(a,b).
connected(b,a).
connected(b,c).
connected(c,b).
```

These facts state that the robot can move between connected locations.
Now define the following rule:

```
can_move(X,Y) :-
    connected(X,Y).
```

You can query Prolog with:

```
?- can_move(a,b).
```

and

```
?- can_move(a,c).
```

Record the results.

Questions

(a) Why does Prolog return `true` for `can_move(a,b)`?

(b) Why does it not establish `can_move(a,c)`?

(c) What is the relationship between the Prolog rule `can_move(X,Y)` and the logical implication

$$\text{Connected}(X, Y) \rightarrow \text{CanMove}(X, Y)?$$

Task 9: Using Prolog to Check a Proposed Plan

Suppose your Python planner has generated the following sequence of movements:

$$\text{Move}(a, b),$$
$$\text{Move}(b, c).$$

Extend your Prolog program with:

```
valid_move(X,Y) :-
    connected(X,Y).
```

Now test:

```
?- valid_move(a,b).
?- valid_move(b,c).
?- valid_move(a,c).
```

The first two queries should succeed, while the third should fail for the knowledge base given above.

Challenge

Now suppose the Python planner proposes:

$$\text{Move}(a, c).$$

Use Prolog to determine whether this proposed action is supported by the warehouse knowledge.

Think About It

The Python program generated the candidate action.
Prolog is being used independently to check whether the action is consistent with the logical description of the warehouse.
Thus we have:

$$\boxed{\text{Generate} \rightarrow \text{Independent verification}}$$

This is the same general architecture discussed in the lecture for using formal reasoning to check AI-generated outputs.

Task 10: Connect Prolog to Logical Reasoning

Consider the Prolog program:

```
wet_road.

slippery :-
    wet_road.

reduce_speed :-
    slippery.
```

Query:

```
?- reduce_speed.
```

Explain why the query succeeds.
Write the corresponding logical reasoning as a sequence of implications.
Your answer should have the form:

$$\boxed{\text{Fact} \Rightarrow \text{Rule} \Rightarrow \text{Rule} \Rightarrow \text{Conclusion.}}$$

Think About It

Prolog is useful for illustrating logical reasoning because the programmer specifies facts and rules, while the Prolog system performs the inference needed to answer a query.
However, Prolog should not be regarded as identical to classical logic.
Its execution mechanism includes operational features such as unification and backtracking, and some Prolog constructs have semantics that differ from classical logical entailment.
For this exercise, focus on the basic idea:

$$\boxed{\text{Facts} + \text{Rules} \rightarrow \text{Inference} \rightarrow \text{Query Answer}}$$

8.2 Reflection

Answer the following questions briefly.

1. What is the difference between a Prolog fact and a Prolog rule?

2. How does a Prolog query correspond to asking whether something follows from a knowledge base?
3. Why might it be useful to use a Prolog program to verify a plan generated by a Python program?
4. What advantage does an independent verifier provide when the original plan was generated with the help of an LLM?

The key idea is:

An AI system can generate a candidate solution, while a separate logical system checks it.